《具身智能与大模型》课程大作业操作指导

LeRobot + LIBERO + SmolVLA 仿真实验

请按照本文档顺序完成环境配置、任务生成、仿真评测与结果整理。

课程统一使用：

```
Python 3.12
LeRobot v0.6.1
LIBERO
SmolVLA
```

请勿在课程实验期间自行升级 LeRobot，也不要修改课堂提供的脚本。LIBERO 需要在 Linux 环境下运行，请自行安装 Ubuntu 系统。

一、准备 Linux 环境

LIBERO 需要在 Linux 环境下运行，请自行安装 Ubuntu 系统。

二、检查 NVIDIA GPU

打开 Ubuntu Terminal：

```
nvidia-smi
```

如果能够正常显示 NVIDIA GPU 型号、驱动版本和显存信息，可以继续。

如果 nvidia-smi 找不到 GPU，请先解决 NVIDIA 驱动问题，再继续后续步骤。

三、更新 Ubuntu

```
sudo apt update
```

然后：

```
sudo apt upgrade -y
```

四、安装基础软件

```
sudo apt install -y git wget curl build-essential ffmpeg
```

安装完成后可以检查：

```
git --version
wget --version
```

五、安装 Miniforge

下载 Miniforge：

```
wget "https://github.com/conda-
forge/miniforge/releases/latest/download/Miniforge3-Linux-x86_64.sh"
```

然后运行安装程序：

```
bash Miniforge3-Linux-x86_64.sh
```

安装过程中按照提示操作。当询问是否更新 shell profile 以自动初始化 conda 时，建议输入 yes。

安装完成后关闭 Ubuntu Terminal，然后重新打开。

检查：

```
conda --version
```

六、创建课程 Python 环境

```
conda create -n lerobot python=3.12 -y
```

激活环境：

```
conda activate lerobot
```

终端前面应出现 (lerobot)。检查 Python 版本：

```
python --version
```

应显示 Python 3.12.x。以后每次重新打开 Ubuntu Terminal，都要先执行：

```
conda activate lerobot
```

七、下载 LeRobot

```
cd ~
git clone https://github.com/huggingface/lerobot.git
```

```
cd ~/lerobot
git checkout v0.6.1
```

检查版本：

```
git describe --tags
```

应显示 v0.6.1。课程期间不要自行升级 LeRobot。

八、安装 LeRobot、SmolVLA 和 LIBERO

确认当前终端前面存在 (lerobot)，并且当前位置是 ~/lerobot。

```
pip install -e ".[smolvla,libero,evaluation]"
```

安装视频保存依赖：

```
pip install imageio imageio-ffmpeg
```

九、检查 LeRobot

```
lerobot-eval --help
```

如果能够正常显示帮助信息，说明 LeRobot CLI 已正确安装。再执行：

```
python -c "import lerobot; print('LeRobot OK')"
```

正常应显示 LeRobot OK。

十、检查 PyTorch 和 CUDA

```
python - <<'PY'
import torch

print("PyTorch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
PY
```

正常情况下应出现 CUDA available: True，并显示 NVIDIA GPU 型号。如果为 False，请先检查 GPU 驱动和 PyTorch 环境（可先尝试重启一次电脑）。

十一、设置 MuJoCo 渲染方式

```
export MUJOCO_GL=egl
```

为了以后不用每次输入，可以执行：

```
echo 'export MUJOCO_GL=egl' >> ~/.bashrc
source ~/.bashrc
```

十二、运行第一次官方 SmolVLA 仿真

第一次运行模型时必须能够正常访问 Hugging Face。

```
conda activate lerobot
cd ~/lerobot
```

运行：

```
lerobot-eval \
  --policy.path=lerobot/smolvla_libero \
  --env.type=libero \
  --env.task=libero_spatial \
  --env.task_ids='[0]' \
  --eval.batch_size=1 \
  --eval.n_episodes=1 \
  --eval.use_async_envs=false \
  --policy.device=cuda \
  '--
env.camera_name_mapping={"agentview_image":"camera1","robot0_eye_in_hand_image":"c
amera2"}' \
  --policy.empty_cameras=1 \
  --output_dir=outputs/libero_test
```

第一次运行会下载 SmolVLA 模型、底层视觉语言模型和 LIBERO assets，时间可能较长，不要中途关闭 Terminal。

十三、确认模型已经下载

检查 SmolVLA 缓存：

```
ls ~/.cache/huggingface/hub/models--lerobot--smolvla_libero
```

正常情况下可以看到 blobs、refs、snapshots。

检查 LIBERO assets：

```
ls ~/.cache/libero/assets
```

如果官方任务能够正常开始执行并生成 rollout 结果，说明基本环境已经配置完成。

十四、以后使用离线模式运行

模型完整下载一次以后，不需要每次连接 Hugging Face。以后在运行仿真脚本时可以使用：

```
HF_HUB_OFFLINE=1 TRANSFORMERS_OFFLINE=1 \
python 脚本名称.py
```

如果离线运行提示找不到模型文件，说明缓存没有完整下载，需要恢复联网并完成一次下载。

十五、放置课堂提供的课程脚本

课堂会提供：

```
libero_custom_task_builder.py
custom_vla_eval.py
```

将两个文件放到 ~/lerobot/ 目录。

```
~/lerobot/libero_custom_task_builder.py
~/lerobot/custom_vla_eval.py
```

不要修改脚本中的模型、相机、图像尺寸或动作处理代码。

十六、生成自己的大作业任务

```
cd ~/lerobot
conda activate lerobot
python libero_custom_task_builder.py
```

按照终端提示：

1. 输入场景编号；

2. 输入自己的英文机器人任务；

3. 等待脚本检查任务并创建文件。

任务示例：

```
Put the wine bottle on top of the cabinet.
Put the milk in the basket.
```

具体允许使用哪些物体和目标，以教师给出的场景说明为准，另外在运行 `libero_custom_task_builder.py` 时，如果输入了不符合该场景的物品，终端会提示哪些物品是符合该场景的。

如果生成成功，程序会创建 .bddl、.json 和 .png 文件，通常位于：

```
~/lerobot/custom_libero_tasks/
```

其中 .png 是任务初始场景预览。此时还没有运行机器人，因此不会生成视频。

十七、检查生成的任务

生成任务后，先检查脚本输出的 Instruction、Goal、Scene 和 Preview。确认输入任务、场景、目标物体和目标位置无误后，再进行 VLA 仿真。

如果任务生成阶段已经报错，不要进入下一步。

十八、运行自己的 SmolVLA 自定义任务

```
cd ~/lerobot

HF_HUB_OFFLINE=1 TRANSFORMERS_OFFLINE=1 \
python custom_vla_eval.py
```

程序会自动显示已经生成的任务。输入希望运行的任务编号并按 Enter。

程序会自动加载 LIBERO 场景、加载 SmolVLA、执行机器人任务、检查是否成功、保存视频和实验结果。运行过程中不要关闭 Terminal。

十九、查看实验结果

运行结束后 Terminal 会显示 SUCCESS 或 FAILURE，并显示结果保存位置。

视频通常位于：

```
~/lerobot/custom_vla_outputs/
```

例如：

```
~/lerobot/custom_vla_outputs/scene_A/put_the_black_bowl_on_the_stove/episode_00.mp
4
```

可以使用：

```
xdg-open 视频完整路径
```

二十、正式实验运行多个 Episodes

正式大作业不要只运行一次。每个最终选择的任务建议至少运行 5 Episodes：

```
HF_HUB_OFFLINE=1 TRANSFORMERS_OFFLINE=1 \
python custom_vla_eval.py --episodes 5
```

程序会依次生成多个 episode 视频，并给出 Success rate。把该结果记录下来。

二十一、查看所有 Rollout 视频

逐个观看视频，记录：

• 机器人是否找到了正确物体；

• 是否成功抓住；

• 是否移动到了正确目标；

• 是否发生碰撞；

• 是否完成放置；

• 第一次失败以后是否重新尝试；

• 是否因为最大 Steps 用完而结束。

如果机器人已经非常接近完成任务，但没有满足 LIBERO 的最终 Goal，正式结果仍然记录为 FAILURE，可以在报告中进一步说明属于接近成功或精细放置失败。

二十二、整理实验结果

每个最终任务至少记录以下内容：

• 自定义任务：输入的英文 Instruction；

• Scene：使用的课程场景；

• Episodes：建议 5；

• Success：成功次数；

• Success Rate：成功率；

• 代表性视频：成功或失败 Rollout；

• 主要失败现象：根据视频填写。

失败现象可以记录为：选择了错误物体、没有抓住、运动过程中碰到其他物体、目标位置判断错误、抬升高度不足、最终放置不准确、第一次失败后无法恢复、达到最大 Steps 后结束等。

不要求所有任务都成功。

二十三、完成大作业报告

1. 实验任务

写出自己设计的自然语言任务，例如：

```
Put the milk in the basket.
```

2. 实验环境

说明使用 LeRobot v0.6.1、LIBERO 和 SmolVLA。

3. 实验次数

例如：5 Episodes。

4. 实验结果

```
Successful Episodes: 2
Total Episodes: 5
Success Rate: 40%
```

5. Rollout 结果

给出代表性视频截图或实验视频。

6. 失败分析

根据实际视频说明失败发生在哪个阶段。不要只写“模型失败”。

示例：模型正确识别并抓取了牛奶，也向篮子移动，但由于机械臂抬升高度不足，牛奶被篮子边缘挡住，最终未满足任务目标。

7. 总结

根据实验结果说明 SmolVLA 在该自定义任务上的表现以及主要局限。

二十四、常见问题

1. conda: command not found

Miniforge 安装完成后关闭 Ubuntu Terminal，再重新打开。如果仍然没有：

```
source ~/miniforge3/etc/profile.d/conda.sh
conda init
```

然后重新打开 Terminal。

2. Hugging Face 下载超时

```
curl -I --max-time 15 https://huggingface.co
```

如果 Terminal 无法访问，需要先解决网络问题。

3. 自定义任务失败

自定义任务失败本身可以是大作业实验结果，可作为大模型能力局限案例进行分析。

二十五、每次重新打开 Ubuntu 后的启动流程

环境安装完成以后，以后不需要重复安装。每次打开 Ubuntu，只需要：

```
conda activate lerobot
cd ~/lerobot
```

生成新任务：

```
python libero_custom_task_builder.py
```

运行任务：

```
HF_HUB_OFFLINE=1 TRANSFORMERS_OFFLINE=1 \
python custom_vla_eval.py --episodes 5
```

完成后查看 ~/lerobot/custom_libero_tasks/ 和 ~/lerobot/custom_vla_outputs/ 中的任务文件、实验结果和 rollout 视频。